

Expressions régulières



Motifs, recherches, filtrages

- repérer dans un texte toutes les occurrences d'une chaîne de caractères donnée et de les remplacer à volonté par une autre ;
- un *motif* est un ensemble d'objets possédant une propriété reconnaissable.

Expressions régulières

- Ce sont des modèles créés à l'aide de caractères ASCII permettant de manipuler des chaînes de caractères
- Il s'agit d'un système très puissant permettant de retrouver un mot, une phrase... dans un texte, ressemblant au modèle que l'on a construit



Expressions régulières

- servent à identifier une chaîne de caractère répondant à un certain critère.
- l'avantage est qu'avec une seule commande on peut réaliser un grand nombre de tâches

Applications

- édition de toutes sortes de textes et programmation
 - font gagner à leur utilisateur un temps considérable pour réaliser des tâches de recherche et de substitution
 - permettent d'analyser la nature d'une information, le contenu d'une variable ou d'un flux de données
 - permettent de détecter la présence de telle ou telle information dont on connaît la structure mais dont le contenu n'est pas déterminé par avance



Expressions régulières

- les ER constituent le langage partagé par tous les outils de traitement de données.
- les spécifications du langage varient cependant d'un outil à l'autre
- apprendre à utiliser les expressions régulières consiste d'abord à maîtriser un langage, c'est-à-dire à découvrir cet ensemble d'opérateurs et leur signification.

Expressions régulières

- Dans une expression régulière certains caractères ont un sens spécial, les autres désignent le caractère lui-même

[éE]tiquettes?

- Les ER permettent de rechercher des occurrences (c'est-à-dire une suite de caractères correspondants à ce que l'on recherche)
- L'ER en elle-même est donc une chaîne de caractères contenant
 - des caractères standards
 - Des caractères spéciaux ou métacaractères
 - [], (), |, etc.

^

début de ligne

[^

opérateur d'exclusion : ne contenant pas les caractères suivants

. Le point correspond à n'importe quel caractère.

^ Indique un commencement de segment mais signifie aussi "contraire de"

\$ Fin de segment

[xy] Une liste de segment possibles.

Exemple label équivaut à : a, b ou c

(x|y) Indique un segment (ps|ump) équivaut à "ps" OU "UMP"

\d le segment est composé uniquement de chiffre, ce qui équivaut à [0-9].

\D le segment n'est pas composé de chiffre, ce qui équivaut à [^0-9].

\s Un espace, ce qui équivaut à [\t\n\r\f\v].

\S Pas d'espace, ce qui équivaut à [^ \t\n\r\f\v].

\w Présence alphanumérique, ce qui équivaut à [a-zA-Z0-9_].

\W Pas de présence alphanumérique [^a-zA-Z0-9_].

\ Est un caractère d'échappement

\D* zéro ou plus caractères non numériques

\D reconnaît n'importe quel caractère *sauf* un chiffre

\d{3} signifie "reconnaître exactement trois chiffres"

- sépare deux caractères à l'intérieur des crochets, définit un intervalle de car.

[0-9] => un chiffre quelconque entre 0 et 9

[a-z] => une minuscule non accentuée entre a et z



Répétitions

- * répétition de motif 0 ou n fois
- + 1 ou n fois
- ? 0 ou 1 fois, optionalité d'un caractère
- {k} le motif est répété exactement k fois ;
- {k, n} le motif est répété entre k et n fois
- {k,} le motif est répété k fois ou plus

19[0-9]{2}

- () permettent d'indiquer comment les éléments de l'expression doivent être groupés.

Fuchs, (C \. | Catherine) =

- $A\{2\}$
- $BA\{1,9\}$
- $BRA\{,10\}$
- $VO\{1,\}$
- $GR(.) + S$
- $GR(.)?S$
- $M(.) + (O) + N$
- $M(.) + ([a-z]) + N$
- $M(.) + ([A-Z]) + N$
- $\wedge!$

- `import re`
- `re.search(r"expr_reg", "string")` # `r` est comme `\` : déspecialise tout ce qu'il y a dedans

Balaie la chaîne (string) à la recherche d'un emplacement du pattern. Si l'expression est trouvée, la fonction renvoie un objet symbolisant l'expression recherchée, elle renvoie *None* si aucune position dans la chaîne ne correspond au motif

```
>>>re.search(r"abc", "abcdef")
```

```
>>>re.search(r"abc", "abacadaef")
```

```
>>>re.search(r"abc", "abacadaef")
```

```
>>>re.search(r"abc*", "ab")
```

```
>>>re.search(r"abc", "abccc")
```

```
>>>re.search(r"chat*", "chateau")
```

- Si l'expression est trouvée dans la chaîne, un objet de la classe `_sre.SRE_Match` est renvoyé. Si l'expression n'est pas trouvée, la fonction renvoie *None*.

- `re.match`

Si 0 ou plusieurs caractères au début de la chaîne correspondent au modèle d'expression régulière, il renvoie un objet correspondant. Renvoie *None* si la chaîne ne correspond pas au modèle.

- Vérifier si une expression est contenue dans une chaîne :

```
>>> if re.match(expression, chaine) is not None :
```

ou

```
>>> if re.match(expression, chaine) :
```

search() vs. match()

- `re.match ()` vérifie une correspondance seulement au début de la chaîne
- `re.search ()` recherche une correspondance n'importe où dans la chaîne.

```
>>> re.match("c", "abcdef") # No match
>>> re.search("c", "abcdef") # Match
<_sre.SRE_Match object at ...>
```

- Les expressions régulières commençant par '^' peuvent être utilisées avec `search ()` pour restreindre la correspondance au début de la chaîne:

```
>>> re.match("c", "abcdef") # No match
>>> re.search("^c", "abcdef") # No match
>>> re.search("^a", "abcdef") # Match
<_sre.SRE_Match object at ...>
```

- Notez cependant qu'en mode MULTILINE, `match ()` ne correspond qu'au début de la chaîne, alors que `search ()` avec une expression régulière commençant par '^' correspondra au début de chaque ligne.

```
>>> re.match('X', 'A\nB\nX', re.MULTILINE) # No match
>>> re.search('^X', 'A\nB\nX', re.MULTILINE) # Match
<_sre.SRE_Match object at ...>
```

des groupes `m.group()`

- Si aucun parenthésage n'a été effectué au sein de la `regEx`, utiliser `m.group(0)` permettra d'avoir accès à la chaîne de caractères reconnues.
- Si un parenthésage a été effectué, il faut utiliser la position de l'élément entre parenthèses pour l'appeler (dans le cas où il a bien été reconnu).

search() vs. match()

```
texte = "Pierre 123 %*$ ?."
```

```
m = re.search("\d+",texte)
```

```
print(m,"\n")
```

```
print(m.group(0))
```

```
print(m.start(0))      #index du premier caractère reconnu
```

```
print(m.end(0))        #index du dernier caractère reconnu
```

```
print(m.span(0))       #index de l'intervalle reconnu
```

```
=>
```

```
<re.Match object; span=(7, 10), match='123'>
```

```
123
```

```
7
```

```
10
```

```
(7, 10)
```

Remplacer *search* par *match*

Faire le tri entre les cas où une correspondance est trouvée et ceux où il n'y a pas de correspondance

```
try : # on essaye d'afficher la chaîne de caractère correspondant au motif  
    print(m2.group(0))  
  
except AttributeError: # s'il n'y en a pas, on évite l'erreur  
    print("Aucune correspondance, désolée !")
```

Aucune correspondance, désolée !

```
if m2: # on teste l'existence d'une correspondance  
    print(m2.group(0))  
  
else: # s'il n'y a pas de correspondance  
    print("Rien, désolée !")
```

Rien, désolée !

findall()

`findall ()` marque toutes les occurrences d'un motif sous forme d'une liste de chaînes. La chaîne est analysée de la gauche vers la droite, et les correspondances sont renvoyées dans l'ordre où elles sont trouvées.

```
re.findall("([0-9]+)", "Bonjour 111 Aurevoir 222")
```

- Trouver tous les adverbes dans un texte

```
>>> text = "He was carefully disguised but captured quickly by police."  
>>> re.findall(r"\w+ly", text)  
['carefully', 'quickly']
```

- `re.compile`

```
prog = re.compile(pattern)
```

L'utilisation de `re.compile ()` est très efficace lorsque l'expression sera utilisée plusieurs fois dans un même programme car il la sauvegarde comme l'objet

Quelques exemples

```
>>> pattern = re.compile("d")
>>> pattern.search("dog")      # Match at index 0
<_sre.SRE_Match object; span=(0, 1), match='d'>
>>> pattern.search("dog", 1)   # No match; search doesn't include the "d"
```

```
>>> pattern = re.compile("o")
>>> pattern.match("dog")       # No match as "o" is not at the start of "dog".
>>> pattern.match("dog", 1)    # Match as "o" is the 2nd character of "dog".
<_sre.SRE_Match object; span=(1, 2), match='o'>
```

- Ecrire une fonction qui supprime tout ce qui ressemble à un suffixe
- Trouver le suffixe d'un mot
- Diviser le mot en racine et en suffixe.

re.finditer

produit des objets de correspondance pour toutes les correspondances non chevauchantes de l'expression régulière sur la chaîne *string* qui est analysée de la gauche vers la droite, et les correspondances sont renvoyées dans l'ordre où elles sont trouvées.

```
texte = "M. Dupont et Mme. Durand ne sont pas venus hier. Mlle. Perrin était là, en revanche. Signé M."

regEx_civilite = re.compile("(Mm?l?l?e?)\s+([A-Z])")

for m in re.finditer(regEx_civilite, texte):
    print(m.group(1), "***", m.group(2))
```

```
M *** D
Mme *** D
Mlle *** P
```

```
texte = "Le 14 avril 1995 et le 10 mai 1992. Le 25 septembre 19991."

regEx = re.compile("(\d+) (avril|mai) (\d+)")

for m in re.finditer(regEx, texte):
    print(m.group(1), m.group(2), m.group(3))
    #print(m.group(0))
```

```
14 avril 1995
10 mai 1992
```

- Segmenter une phrase

```
>>> txt = """"When I'M a Duchess,' she said to herself, (not in a very hopeful  
tone ... though), 'I won't have any pepper in my kitchen AT ALL. Soup does very  
... well without--Maybe it's always pepper that makes people hot-  
tempered,'..."""
```

- Faire de même avec une expression régulière

```
>>> re.split(r' ', txt)
```

```
['"When"', 'I'M"', 'a', 'Duchess,', '"', 'she', 'said', 'to', 'herself,', '(not', 'in', 'a', 'very',  
'hopeful', 'tone\nthough),', '"I"', 'won't"', 'have', 'any', 'pepper', 'in', 'my',  
'kitchen', 'AT', 'ALL.', 'Soup', 'does', 'very\nwell', 'without--Maybe', 'it's',  
'always', 'pepper', 'that', 'makes', 'people', 'hot-tempered,..."']
```

```
>>> re.split(r'[ \t\n]+', txt)
```

```
['"When"', 'I'M"', 'a', 'Duchess,', '"', 'she', 'said', 'to', 'herself,', '(not', 'in', 'a', 'very',  
'hopeful', 'tone', 'though),', '"I"', 'won't"', 'have', 'any', 'pepper', 'in', 'my',  
'kitchen', 'AT', 'ALL.', 'Soup', 'does', 'very', 'well', 'without--Maybe', 'it's',  
'always', 'pepper', 'that', 'makes', 'people', 'hot-tempered,..."']
```

- Utiliser une abréviation intégrée, \s, qui signifie tout caractère d'espace : re.split(r'\s+', txt)

- utilisez `\W` dans une expression régulière simple pour segmenter l'entrée sur autre chose qu'un caractère de mot:
- ```
>>> txt = """When I'M a Duchess,' she said to herself, (not in a very hopeful tone ... though), 'I won't have any pepper in my kitchen AT ALL. Soup does very ... well without--Maybe it's always pepper that makes people hot-tempered,'..."""
```

```
>>> re.split(r'\W+', txt)
```

```
['', 'When', 'I', 'M', 'a', 'Duchess', 'she', 'said', 'to', 'herself', 'not', 'in', 'a', 'very', 'hopeful', 'tone', 'though', 'I', 'won', 't', 'have', 'any', 'pepper', 'in', 'my', 'kitchen', 'AT', 'ALL', 'Soup', 'does', 'very', 'well', 'without', 'Maybe', 'it', 's', 'always', 'pepper', 'that', 'makes', 'people', 'hot', 'tempered', '']
```

## **Segmenter le texte avec deux solutions :**

texte = "Hier, il a fait vraiment beau ! Est-ce que tu penses que ce sera pareil aujourd'hui ?"



# Remplacer une expression

- **La fonction *sub* du module *re***

```
>>>re.sub(r"ab", r"ab ", "abcdef") #ajouter une espace après ab
'ab cdef'
```

```
>>>re.sub(r"(ab)", r"\1 ", "abcdef")#mémoriser un motif
'ab cdef'
```

```
texte = "La petite fleur aime le soleil."
```

```
nouveau_texte = re.sub("a","***",texte)
print(nouveau_texte)
```

```
nouveau_texte2 = re.sub("[aeiuo]","V",texte)
print(nouveau_texte2)
```

```
nouveau_texte3 = re.sub("[aeiuo]{2,}","V",texte)
print(nouveau_texte3)'
```

# Remplacer à l'aide d'une fonction

```
def remplace_civilite(m):
 """Permet de remplacer les abréviations de civilités par leur nom complet."""
 civilite = str()
 if m.group(1) == "M":
 civilite = "Monsieur"
 elif m.group(1) == "Mme":
 civilite = "Madame"
 elif m.group(1) == "Mlle":
 civilite = "Mademoiselle"
 return civilite+" "+m.group(2)

texte = "M. Dupont et Mme. Durand ne sont pas venus hier. Mlle. Perrin était là, en revanche. Signé M."

regEx_civilite = re.compile("(Mm?l?l?e?)\s+([A-Z])")

for m in re.finditer(regEx_civilite,texte):
 print(m.group(1),m.group(2))

nouv_texte = re.sub(regEx_civilite,remplace_civilite,texte)

print(nouv_texte)
```

- re.DEBUG

affiche des informations sur le débogage de l'expression compilée

- re.I , re.IGNORECASE
- re.M, re.MULTILINE

# ER : memo

- `re.compile()` : pour compiler et enregistrer un motif
- `re.match()` : cherche une correspondance au début de la chaîne
- `re.search()` : chercher la première correspondance
- `re.finditer()` : itérateur qui cherche toutes les correspondances
- `re.sub()` : pour remplacer
- `re.split()` : pour découper
- `match.group()` : pour appeler les séquences mémorisées



# ER mémo

- `re.match(r"ER", "String")`
- `re.search(r"ER", "String")`
- `re.findall("ER", "String")`
- `re.sub(r"ER", r"ER", "String")`
- `regex = re.compile(r"ER")`
- `r` avant `ER` est comme `\` : déspecialise tout ce qu'il a dedans

# Exercice

- Trouvez des adverbes dans une phrase / un texte